

Nomos requirements

2026-09-22

Table of contents

Introduction	2
1. Components at a glance	3
To sum up, in three tiers	4
2. Deployment options	5
Option A — Docker Compose on one VM (simplest, what we use today)	5
Option B — Managed PostgreSQL	5
Option C — Hybrid	5
3. The database	6
3.1 Version and layout	6
3.2 Required extensions	6
3.3 Sizing	7
3.4 Backup	7
4. Database accounts	8
5. Who connects to the database, from where	10
6. The worker (Nomergon)	12
7. Embeddings on the worker's GPU	14
8. LLM endpoint	15
8.1 How the software talks to an LLM	15
8.2 What the LLM is used for	15
8.3 Models	16
8.4 If self-hosting is not possible	16
9. Observability (Phoenix) and the database console (pgAdmin)	17
9.1 Arize Phoenix — we recommend hosting it centrally	17
9.2 pgAdmin — genuinely optional	17
9.3 Credentials	17
10. Workstation requirements (analysts)	18
11. Security and data protection summary	18
12. decision checklist	19
13. Minimal viable deployment	19

Audience: the JRC / EUROMOD IT team.

Purpose: describe what has to be hosted for the N6mos prototype to run, what is *mandatory* versus *nice to have*, and which decisions we need from IT (Docker or not, managed PostgreSQL or not, self-hosted GPU or not).

Introduction

Nómos is a prototype, built under a JRC expert contract, that partially automates a job currently done by hand: updating the fiscal parameters of **EUROMOD** — the EU-wide tax-benefit microsimulation model — when national legislation changes. Every year, country teams read finance acts, social-security decrees and tax codes in their own language, find the figures that changed (tax bands, benefit amounts, contribution rates), and transcribe them into EUROMOD’s parameter files. Nómos does the reading and the proposing; a human still does the deciding.

Concretely, the system works in four stages. It **ingests** official national legislation from the public portals (Légifrance, e-tar.lt, BOE, eISB...), archiving each source document and splitting it into searchable, translated, vector-indexed passages. An **agentic workflow** then takes one EUROMOD parameter at a time, retrieves the relevant legal passages, and asks an LLM to propose the updated value — with a mandatory verbatim quote of the article it comes from, checked against the stored text, so a proposal that cannot be traced to real legal wording is rejected automatically. A **desktop validation UI** presents each proposal, its citation and its source text to an analyst, who accepts or rejects it. Finally, an **evaluation pipeline** scores the whole chain against a golden set of known-correct parameters, which is how we compare LLMs and measure whether the system is actually good enough to be useful.

Nómos never writes into EUROMOD. It is export-first and human-gated: nothing leaves the review queue without an explicit human Accept.

Why a central database rather than files on a laptop. All four stages read and write the same PostgreSQL instance, and this is a design decision:

- The corpus is **expensive to build and shared**. Ingesting, translating and embedding a national corpus costs hours of compute; it must be done once for everyone, not once per analyst.
 - Citations are **pointers into that corpus** (a chunk id plus character offsets). A proposal reviewed six months later must resolve to the exact same text — which requires a single authoritative store, not copies drifting apart.
 - The **audit trail is a contractual deliverable**. Every human decision is appended to a central, append-only table linked to the proposal and its evidence. It is the one artefact in the system that cannot be regenerated by re-running anything.
 - **Reproducibility and comparability**. Evaluation results, model comparisons and KPIs are only meaningful if every run drew on the same corpus and the same parameter store.
-

1. Components at a glance

Component	What it is	Where it can run	Needed for a minimal deployment?
1 Legislation database (Nomotheca)	PostgreSQL 17 + pgvector. Holds legal texts, chunks, embeddings, the parameter store and the evaluation results	Central server / managed service	YES — mandatory, shared
2 Ingestion pipeline (nomotheca-ingest)	Python CLI. Fetches legislation from official national sources, snapshots, parses, chunks, loads, embeds, translates	Inside the worker (component 10)	Yes
3 Agentic workflow (nomoscope-workflow)	Python CLI. Proposes parameter updates with an LLM	Inside the worker (component 10)	Yes
4 Validation UI (Nomoscope)	Tauri 2 desktop application (Rust + Svelte) — an <code>.exe</code> / AppImage, <i>not</i> a web app	Analyst workstation	Yes — it is the human-in-the-loop step
5 Evaluation pipeline (nomokrisis-eval)	Python CLI, scores the workflow against a golden set	Inside the worker (component 10)	Optional (project internal)
6 Arize Phoenix	LLM observability (traces, token counts, cost, energy)	One small container — better centrally hosted , dev machine as fallback	Not for the system to <i>run</i> , but see §9
7 pgAdmin	Database web console	One small container — better centrally	No — droppable

Component	What it is	Where it can run	Needed for a minimal deployment?
		hosted , dev machine as fallback	
8 Embeddings model	BGE-M3, 1024-dim vectors	Inside the worker, on its GPU (CPU fallback) — never on a workstation	Yes, see §7
9 LLM endpoint	Open-weight model behind an OpenAI-compatible API	GPU server (preferred) or external API — reached only from the worker	Yes, in one form or another
10 Worker (Nomer ^{gon})	One container that executes every job the UI or a developer submits (ingestion, embedding, translation, workflow runs, evaluation). The only process that holds model credentials or loads a model	Server VM with an NVIDIA GPU (CPU fallback works, slowly)	YES — mandatory, shared

To sum up, in three tiers

- **Must have — components 1 and 10.** Without the database there is no system; without the worker nothing runs, because the workstation executes nothing (§6).
- **Should have — a GPU for the worker, component 9, and component 6.** The GPU (§7) and a self-hosted LLM (§8) are what make the system fast and keep all data inside the JRC perimeter. **Phoenix (§9) is the one piece of “tooling” we would argue for hosting centrally**, because it holds the project’s evidence base, not just developer convenience — reasons in §9.
- **Nice to have — component 7.** Components 2 and 5 are no longer separate deployables: they ship inside the worker image.

2. Deployment options

Option A — Docker Compose on one VM (simplest, what we use today)

A single `docker-compose.yml` at the repo root starts PostgreSQL + pgvector, pgAdmin, Phoenix and the worker, and auto-applies the schema. One `docker compose --profile gpu up -d` and the server side is running.

- Requires: one Linux VM, Docker + Docker Compose, ~4 vCPU / 16 GB RAM / 100 GB disk, and the NVIDIA container toolkit if the VM has a GPU (the worker falls back to CPU otherwise).
- Pros: fastest, reproducible, exactly what has been tested; schema and extensions come pre-provisioned in the `pgvector/pgvector:pg17` image.
- Cons: you own the backups and the patching of a self-hosted database.

Option B — Managed PostgreSQL

The database is a managed service (AWS RDS/Aurora, or an internal DBaaS); we apply the schema over the network with `psql`. The worker still needs a container host or a Linux VM with `uv` (a single static binary); the UI is a signed desktop installer.

- This works, **provided the required extensions are available** — see §3.2. This is the one blocking question for Option B.
- pgAdmin can simply be dropped. Phoenix still needs somewhere to run — a small container host, or a dev machine as fallback (§9.1).

Option C — Hybrid

Managed PostgreSQL (backed up, monitored) + one Linux VM with a GPU running the worker and the Phoenix container + desktop UI on the analysts' machines + pgAdmin dropped in favour of IT's own database console.

3. The database

3.1 Version and layout

- **PostgreSQL 16 or later** (tested on 17).
- Two logical databases on the same instance:

Database	Contents	Who writes
<code>legislation</code>	<code>public</code> schema — legal instruments, texts, chunks, embeddings, fetch snapshots. <code>params</code> schema — the EUROMOD parameter store, proposals and extraction runs. <code>eval</code> schema — evaluation results	ingestion, workflow, evaluation
<code>phoenix</code>	LLM trace storage for Arize Phoenix	Phoenix only

3.2 Required extensions

Extension	Why
<code>vector</code> (<code>pgvector</code> ≥ 0.7)	Vector search. Version 0.7+ is required — we store <code>halfvec(1024)</code> (fp16) with an HNSW index; earlier versions do not have <code>halfvec</code>
<code>pg_trgm</code>	Fuzzy matching of legal citations / identifiers
<code>btree_gist</code>	UUID equality inside GiST exclusion constraints (temporal validity)
<code>unaccent</code>	Diacritics-insensitive full-text search
<code>ltree</code>	Legal hierarchy paths (book $\rightarrow$ title $\rightarrow$ chapter $\rightarrow$ article)

`CREATE EXTENSION` normally requires a superuser or a specially privileged role. On managed services the extensions must usually be **allow-listed** first (`shared_preload_libraries` / parameter group on RDS). We need either that allow-listing plus a role that can create them, or a DBA to run `Nomotheca-RAG/db/schema.sql` once for us.

The schema also creates **per-language full-text search configurations** (French, Dutch, German, Spanish, English, Lithuanian, Irish ...) built on the standard PostgreSQL stemmers — no third-party dictionary is needed.

3.3 Sizing

Today's pilot corpus (partial, 3 countries, ~70 instruments) is **228 MB**:

Table	Size	Note
<code>chunks</code>	64 MB	retrieval units + FTS index
<code>unit_texts</code>	57 MB	original + machine-translated text
<code>embeddings</code>	48 MB	8 845 vectors, halfvec(1024) + HNSW
<code>fetch_snapshots</code>	17 MB	raw archived payloads (provenance)

Extrapolating to the 27 countries fully ingested, with English translations and embeddings, we expect **70 GB**. **Provision 100 GB with room to grow**, and expect the vector index to want to sit in RAM: **8–16 GB RAM** on the instance is comfortable, 4 vCPU is plenty. There is no high write throughput — ingestion is batch, the workflow is mostly reads.

3.4 Backup

Standard nightly backup + PITR is more than enough. Nothing here is unrecoverable in principle (the corpus can be re-ingested from official sources), but re-ingestion is slow and re-embedding costs GPU hours, so a normal backup policy saves real time. The one genuinely irreplaceable artefact is `params.review_decisions`, the append-only human-decision audit log — it is the only table that cannot be regenerated by re-running the pipeline, so it is the reason a backup policy matters here at all.

4. Database accounts

We currently use one all-powerful `jrc` user because it is a local prototype. **For a real deployment we recommend separate least-privilege roles**, one per component. This also makes the audit trail meaningful — you can tell from `pg_stat_activity` which component did what.

Role	Rights	Used by
<code>nomos_admin</code>	Owner of the schemas, runs migrations	DBA / dev team, manual
<code>nomos_worker</code>	read/write on <code>public</code> , <code>params</code> , <code>eval</code> and <code>ops</code> — the only role that writes the corpus, the embeddings, the parameter store, the review queue and job status	the worker (component 10)
<code>nomos_reviewer</code>	A <i>group</i> role, no login. read-only on <code>public</code> , <code>params</code> , <code>eval</code> ; INSERT on <code>params.review_decisions</code> ; INSERT on <code>ops.jobs</code> ; UPDATE of <code>ops.jobs.cancel_requested</code> on its own rows (row-level security); UPDATE of the verification columns of <code>eval.golden_cases</code>	validation UI (component 4), via the analyst logins below
<code><analyst></code>	One login role per analyst , member of <code>nomos_reviewer</code>	each analyst's UI. Few analysts, and every decision and job then carries <code>current_user</code>
<code>phoenix</code>	owner of the <code>phoenix</code> database only	Phoenix container (component 6)

The UI writes exactly two things: a human decision and a job request. Everything else it does is a read. This is what makes per-analyst logins cheap to grant and worth having: the audit trail says who decided, who submitted, and who verified a golden case, from the database's own identity rather than from an environment variable.

Sketch of the grants (to be run by a DBA after the schema is applied):

```
CREATE ROLE nomos_reviewer NOLOGIN;
GRANT CONNECT ON DATABASE legislation TO nomos_reviewer;
GRANT USAGE ON SCHEMA public, params, eval, ops TO nomos_reviewer;
GRANT SELECT ON ALL TABLES IN SCHEMA public, params, eval, ops TO nomos_reviewer;
GRANT INSERT ON params.review_decisions, ops.jobs TO nomos_reviewer;
```

```
GRANT UPDATE (cancel_requested) ON ops.jobs TO nomos_reviewer;
GRANT UPDATE (verified, verified_by, verified_at) ON eval.golden_cases TO
nomos_reviewer;
-- row-level security on ops.jobs: an analyst cancels only what they submitted
CREATE ROLE alice LOGIN PASSWORD '...' IN ROLE nomos_reviewer;
```

The ready-to-run version of these grants is `Nomergon-worker/db/roles.sql`.

Note on “scout”. The optional scout feature (`WORKFLOW_SCOUT=llm|tavily`) ingests a missing legal act on the fly. It runs inside the worker, which already holds write rights on `public`, so it needs no extra role.

Connection security: all components accept a standard PostgreSQL URL and support TLS (`?sslmode=require`). Please issue **TLS-only** accounts. Credentials are read from environment variables (`WORKFLOW_DATABASE_URL`, `EUROMOD_DATABASE_URL`) — they can come from a vault, a systemd unit, or a `.env` file, whatever fits JRC practice. Nothing is hard-coded.

5. Who connects to the database, from where

This is the part most likely to need a firewall rule, because **the validation UI is a desktop application, not a web app**. It talks to PostgreSQL directly.

Client	Host	Connection	Notes
Validation UI (Tauri desktop app)	Analyst laptop/workstation	PostgreSQL 5432 (TLS)	Nothing else. The UI reads, records decisions, and submits jobs; it spawns no process and holds no model credential
Worker (Nomerigon)	The server VM	PostgreSQL 5432 + outbound HTTPS to official national legislation portals (Légifrance/DILA, eISB, e-tar.lt, BOE, ...) + HTTPS to the LLM endpoint (and to Tavily if the scout's web search is on)	The only machine that needs the internet and the only one that holds API keys
Developer CLIs (ingest, workflow, eval)	Inside the worker container (docker compose exec worker ...)	as the worker	A command that needs a model runs here; a command that needs only the database runs anywhere
Phoenix	Small VM / container host (preferred), or a dev machine	PostgreSQL 5432 to the phoenix DB; receives OTLP traces on 6006/4317 from every machine that runs the workflow	See §9.1
pgAdmin	Dev machine	PostgreSQL 5432	Optional

If direct database access from user workstations is not acceptable, the alternatives are a VPN (simplest), a bastion/SSH tunnel, or restricting the desktop UI to a small group of analyst machines on a dedicated subnet. **Please tell us which model applies** — it does not change the code, only the connection string.

There is deliberately **no HTTP API** between the UI and the worker: the database is the bus. A job is a row the UI inserts and the worker claims; progress is rows the UI polls (§6). One network path to approve instead of two. Should IT refuse workstation-to-Postgres altogether, a thin API in front of the same tables is the fallback.

6. The worker (Nomergeron)

Everything that calls a model, loads a model, or fetches from the internet runs in **one process on the server**, the worker. The desktop UI executes nothing: it inserts a **job** into the `ops.jobs` table and polls the job's status and log lines from `ops.job_events`. The worker claims queued jobs (`SELECT ... FOR UPDATE SKIP LOCKED`), runs each one as a subprocess of the existing command-line tools, and streams their progress back into the database.

Job type	What it does	Model needed
<code>workflow</code>	One or more agentic runs (country, parameter, system year)	LLM + BGE-M3
<code>ingest</code>	Fetch, parse, chunk and load an instrument, a citation, a contributed document	none (internet)
<code>embed</code>	Embed every chunk not yet embedded	BGE-M3 on the GPU
<code>translate</code>	Translate every legal text not yet in English	LLM
<code>translate-params</code>	Translate EUROMOD parameter metadata into a national language	LLM
<code>encode</code>	Encode one search query for the UI's semantic search — a priority lane, model kept warm	BGE-M3
<code>eval</code>	Score the workflow against the golden set	LLM + BGE-M3
<code>draft-golden</code>	Draft golden cases from the OpenFisca corpus	LLM
<code>impact</code>	The EcoLogits environmental-impact report over Phoenix traces	none

What this buys:

- **API keys exist on one machine.** The worker's environment (a vault, a systemd `EnvironmentFile`, or the compose `.env`) is the only place an LLM key or a gateway token lives. Workstations hold their database login and nothing else.
- **The GPU is shared.** BGE-M3 runs once, on the server's GPU, for every analyst. No model download, no PyTorch, no Python on a workstation.
- **Batches have a home.** Embedding and translation are resumable (each run picks up what is missing), so a night-time batch is simply a job submitted in the evening. For now it is submitted by hand — from the UI, or `docker compose exec worker nomergon submit embed` — and a scheduler is a later, separate decision.

- **Nothing lives on a laptop.** The review queue, the decisions, evaluation runs and the golden set are rows in the database. The one file a workstation ever writes is the export of accepted values for EUROMOD, one file per country, saved where the analyst chooses.

Sizing: one job at a time (embedding, translation and workflow runs compete for the same GPU), plus the always-on `encode` lane. **One NVIDIA GPU with 8–16 GB** is ample for BGE-M3; the worker also runs on CPU, roughly twenty times slower for embedding. RAM 16 GB. Disk: the image (~6 GB, it includes a headless Chromium for the Légifrance fallback) plus the model cache (~2.5 GB) on a persistent volume. The container needs the NVIDIA container toolkit on the host; in dev it is `docker compose --profile gpu up -d (--profile cpu` without a GPU).

7. Embeddings on the worker's GPU

Model: **BAAI/bge-m3**, 1024-dimensional, multilingual (covers all the pilot languages — this is why it was chosen over English-centric encoders).

Embeddings are needed in two places:

1. **Batch** — every ingested chunk is embedded once. This is the heavy job: hundreds of thousands of chunks for a full 5-country corpus.
2. **Query time** — every workflow run encodes its query, and the UI's semantic search encodes what the analyst types (an **encode** job, answered by the worker within a fraction of a second).

Both happen **inside the worker**, on its GPU, with the model kept loaded. Workstations need neither the model nor the ML dependencies, and there is exactly one encoder version for everybody, so embeddings stay comparable (mixing encoder versions silently degrades retrieval).

Batch embedding of a national corpus takes hours on a GPU and days on CPU.

Requirement	Value
Model	BAAI/bge-m3 (Apache-like MIT licence, freely redistributable)
VRAM	~4 GB fp16 (~2.3 GB weights + activations at 8 k context). Any 8–16 GB GPU is ample
Serving stack	Loaded in-process by the worker (PyTorch on CUDA; OpenVINO/CPU fallback)
Interface	none — the worker is the only client

If JRC already runs a shared embeddings service (Hugging Face TEI, Infinity, vLLM — all expose an OpenAI-compatible `/v1/embeddings`), the worker can use it instead of its own GPU: the encoder sits behind an `EmbeddingBackend` protocol with one method (`encode(list[str]) -> list[list[float]]`), and an HTTP backend is a small, isolated addition. The endpoint must serve **BAAI/bge-m3** with normalised vectors so it stays comparable with the corpus embedded so far.

Note that embedding dimensions are baked into the schema (`halfvec(1024)`), so a different embedding model would require a schema change and a full re-embed. Sticking to BGE-M3 is the low-friction path.

8. LLM endpoint

8.1 How the software talks to an LLM

Models are configured everywhere as **provider-prefixed strings** (`anthropic/...`, `openai/...`, `azure_openai/...`, `openrouter/...`, `together/...`, plus `mock/...` for key-less offline demos). Nothing in the pipeline knows the provider — endpoint flexibility is a hard contractual requirement, because Activity 5 has to compare several models.

A self-hosted model served behind an OpenAI-compatible API (vLLM, SGLang, TGI, Ollama, LM Studio, or any internal gateway) is consumed through its own `jrc/` prefix, so a run on the internal gateway is never confused with one on OpenAI proper when models are compared — an environment-variable change, **on the worker only**:

```
WORKER_MODELS=jrc/mistral-small-3.2,azure_openai/gpt-5.6-luna      # what
analysts may pick
JRC_LLM_BASE_URL=https://llm.internal.jrc/v1
JRC_LLM_API_KEY=<token issued by the gateway>
```

The worker publishes the models it is configured for; the UI offers that list and nothing else. No analyst ever sees or types a key.

If the internal gateway is not OpenAI-compatible, adding a provider prefix is ~10 lines in `llm.py`.

8.2 What the LLM is used for

Use	Volume	Notes
<code>propose</code> step	1 call per parameter per run	Structured output (JSON schema), long RAG context
<code>critique</code> step	1 call per parameter per run, can use a different/cheaper model	
Bulk translation of non- English legal text to English	This is the volume driver — every ingested legal unit	Segmented ~6 k characters per call; a self-hosted model makes this essentially free
Optional “scout” gap-fill	rare	

Bulk translation is a `translate` job: submitted in the evening, it runs through the night, and an interrupted batch loses nothing because the next one starts from what is still missing.

Requirements: **structured / JSON output support**, a **long context window** (32 k+, RAG prompts carry many legal chunks), and solid **multilingual** ability for EU languages.

8.3 Models

The models EUROMOD already runs are all suitable, and are what we would target first:

Model	Total / active params	VRAM (Q4/FP8)	Comment
Mistral Small 3.2 (24B)	24B	~15 GB	Apache 2.0, strong EU-language coverage. Best fit for a single 24 GB GPU
gpt-oss (20b / 120b)	21B/3.6B · 117B/5.1B	~16 GB (native MXFP4) / ~80 GB	Apache 2.0, MoE → fast. English-centric: check quality on FR/LT/GA legal text before relying on it
MiniMax 2.7	large MoE	multi-GPU	Very long context — useful for the RAG prompts
Llama 3.3 70B	70B	~40–48 GB	⚠ Llama Community Licence (700 M-MAU cap, branding obligations). Covers 8 languages — Lithuanian and Irish are <i>not</i> among them

Alternatives worth knowing about, if EU-language parity turns out to be the binding constraint: **EuroLLM-9B** and **Teuken-7B** are the only open models trained from scratch on all 24 EU official languages, and both run in ~6 GB. See [Nomoscope-agentic-workflow/models_choice.md](#) for the full comparison.

Suggested hardware: **one 24 GB GPU** (RTX 4090 / L4 / A5000) serves Mistral Small 3.2 for a small analyst team. **One 80 GB GPU** (A100/H100) covers gpt-oss-120b or Llama 3.3 70B and comfortable throughput for bulk translation. The same GPU can host the embeddings model alongside — BGE-M3 needs only ~4 GB.

8.4 If self-hosting is not possible

The system also works against external APIs (Anthropic, OpenAI, Azure OpenAI, OpenRouter, Together). This is a **data-governance decision, not a technical one**: the content sent to the model is published national legislation plus EUROMOD parameter metadata — public data, no personal data — but it does leave the JRC perimeter. **Azure OpenAI in an EU region** is the usual middle ground.

9. Observability (Phoenix) and the database console (pgAdmin)

Neither is required for the system to *run*. They are not, however, in the same category, and we would ask IT to treat them differently.

9.1 Arize Phoenix — we recommend hosting it centrally

Phoenix (<http://localhost:6006> in the dev stack) collects an OpenTelemetry trace of every LLM call: the prompt, the retrieved chunks, token counts, latency, cost, and the energy/CO₂ estimate. Three reasons it belongs on IT infrastructure rather than a developer's laptop:

1. **It holds a project deliverable, not just debug output.** Activity 5's multi-model comparison and the environmental-impact KPIs are computed *from these traces*. If the trace history lives on a contractor machine, the evidence base for the final technical report disappears at the end of the contract.
2. **The data is already central anyway.** Phoenix stores everything in the `phoenix` database inside the same PostgreSQL instance (`PHOENIX_SQL_DATABASE_URL`). Only the web container is local today. So hosting it properly is a small increment — the state that needs backing up is already in the database IT is backing up.
3. **Debugging is a shared activity.** When an analyst reports a wrong or missing proposal, diagnosing it means opening that run's trace. With Phoenix on one developer's machine, only that developer can answer the question.

Needs: **one container** (`arizephoenix/phoenix:latest`), ~1 GB RAM, its own database user on the `phoenix` DB, and two ports — 6006 (web UI + OTLP/HTTP) and 4317 (OTLP/gRPC) — reachable from wherever the workflow runs.

Two caveats to plan for:

- **The open-source image has no authentication by default.** Behind a reverse proxy with JRC SSO, or restricted to an internal subnet. The content is prompts over public legal text, so the confidentiality risk is low, but it should not be an open web app.
- It is a fast-moving project — **pin a version tag** rather than tracking `:latest` as our dev compose file does.

9.2 pgAdmin — genuinely optional

pgAdmin (<http://localhost:5050> in the dev stack) is a convenience console with no data of its own. Any PostgreSQL client replaces it — DBeaver, `psql`, or the validation UI's own read-only Database tab. It is also a web console holding cached database credentials, i.e. an attack surface that buys nothing unique.

If IT already provides a database console or a standard client, we use that and drop pgAdmin. We only kept it in the development stack because it ships in the same `docker compose up`.

9.3 Credentials

Both currently run with demo credentials (`admin@euromod.eu` / `admin`, `jrc` / `jrc`). These are development defaults and must be replaced before either is exposed beyond a developer machine.

10. Workstation requirements (analysts)

Item	Requirement
OS	Windows 10/11, macOS, or Linux (Tauri 2 builds for all three; developed on WSL2/Linux)
Validation UI	Installer (~15 MB). On Windows it uses the system WebView2 runtime
Python pipeline	Not needed. Nothing but the UI is installed
Network	PostgreSQL TCP to the database. Nothing else — no LLM endpoint, no internet
Local disk	The installer, plus the EUROMOD export files the analyst chooses to save
GPU	Not needed

Build-side only (dev team): Rust toolchain + Node.js for compiling the UI, and the usual Linux WebKitGTK dev packages.

11. Security and data protection summary

- **Data classification: public.** The corpus is published national legislation, plus EUROMOD parameter values and Country Reports. **No personal data, no microdata, no EU-SILC data** ever enters this system.
- **Outbound traffic:** only the worker talks to the outside — ingestion fetches from official national legislation portals, the workflow calls the LLM endpoint. Workstations talk to PostgreSQL and nothing else. With a self-hosted LLM and a pre-ingested corpus, the system needs **no internet access at all** at run time.
- **The system never writes into EUROMOD.** It is export-first and human-gated: proposals go into a review queue, a human accepts or rejects in the UI, and only accepted values are exported. There is no write path into EUROMOD parameter files.
- **Audit trail:** every proposal carries its citations (chunk id + character offsets into the source text), and every human decision is appended to `params.review_decisions` — append-only, and linked back to the proposal and the Phoenix trace it was taken on. This is a contractual property; it lives in the database precisely so it is protected by the same backup policy as everything else (§3.4).
- **Secrets:** API keys exist on the worker only, from environment variables (a vault, a `systemd EnvironmentFile`, or the compose `.env`). A workstation holds one secret: the analyst's own database password, in the OS credential store.
- **Least privilege:** the UI's role can read, append a decision, and submit a job. It cannot alter the corpus, a proposal, or anyone else's job (§4).

12. decision checklist

Question	Our preference
1 Is Docker allowed on a server VM?	Yes → Option A, zero setup work
2 If not: managed PostgreSQL 16+ available?	Fine — see §3
3 Can pgvector ≥ 0.7, pg_trgm , btree_gist , unaccent , ltree be enabled?	Blocking for Option B. Needs allow-listing on most managed services
4 Who runs <code>schema.sql</code> — us with a privileged role, or a DBA?	Either
5 Dedicated DB accounts : one worker role, one login per analyst, TLS-only?	Yes, per §4 — a handful of analysts
6 Can analyst workstations reach the DB on 5432 (directly / VPN / bastion)?	VPN is fine; we need to know which
7 A Linux VM with an NVIDIA GPU (8–16 GB) and the NVIDIA container toolkit for the worker?	Strongly wanted ; CPU-only works but embeds ~20× slower
8 Is a self-hosted LLM endpoint possible, OpenAI-compatible? Which of Mistral Small 3.2 / gpt-oss / MiniMax 2.7 / Llama 3.3 70B are already served?	Strongly wanted ; Mistral Small 3.2 first
9 If not: is an external LLM API (Azure OpenAI EU, Anthropic, ...) authorised for public legal text?	Needed for Activity 5's multi-model comparison either way
10 Can Phoenix be hosted centrally (one container, ~1 GB RAM, behind SSO)?	Yes — it holds the Activity 5 evidence base, §9.1. Fallback: dev machine
11 Do you already provide a database console ? Then we drop pgAdmin	Either
12 Outbound HTTPS from the worker VM to national legislation portals (and the LLM endpoint)?	Required — it is the only machine that needs it
13 Backup policy on the <code>legislation</code> database?	Standard nightly + PITR

13. Minimal viable deployment

1. One PostgreSQL 17 instance with pgvector, one database, the roles of §4. (*IT*)
2. One Linux VM running the worker container (GPU if available, CPU otherwise) and Phoenix. (*IT provides the VM, we deploy*)
3. The desktop UI on the analysts' machines. (*us*)
4. LLM: whatever endpoint is authorised, configured on the worker.

That is enough to run the full pilot end-to-end. Every other item in this document is an improvement in speed, security or operability, not a precondition.